



Distributed Out-of-Core MergeSort

Parallel and Distributed Systems: Paradigms and Models (SPM)

Samiullah Khairy

Student ID: 684042

Prof. Massimo Torquati

Date: January 2026

Abstract

This report details the design and implementation of a scalable out-of-core MergeSort for large datasets with variable-size records. The project targets both shared-memory and distributed-memory platforms using OpenMP, FastFlow, and MPI. A specific focus is placed on memory-constrained environments where the input size exceeds available RAM. Performance results confirm the scalability of the FastFlow pipeline design and show that the hybrid MPI+OpenMP approach scales well up to moderate node counts, after which the global merge phase dominates.

1 Introduction

This project focuses on the design and implementation of a scalable, external merge sort algorithm capable of sorting datasets that exceed the available RAM of a single node. The solution targets a cluster environment, leveraging both shared-memory parallelism (OpenMP, FastFlow) and distributed-memory parallelism (MPI). The work was carried out in the context of the Parallel and Distributed Systems: Paradigms and Models (SPM) course and targets both shared-memory and distributed-memory platforms.

The core objective was to sort binary files containing variable-length records. Each record consists of an 8-byte sorting key, a 4-byte length indicator, and a variable-size payload. The implementation adheres to strict memory budgets, ensuring that large datasets are handled via external sorting techniques (sorting chunks in memory, flushing to disk, and merging sorted runs).

The primary objectives were:

- To implement a sequential version to be treated as the baseline for the parallel version
- To implement efficient single-node parallel versions using **OpenMP** and **FastFlow**;
- To design a **hybrid distributed solution** combining **MPI + OpenMP**;
- To evaluate performance and scalability on the SPM cluster under realistic memory constraints.

The emphasis throughout the project is on correctness, clarity of design, and minimizing parallelization overhead, rather than on aggressive low-level optimizations.

2 Problem Description and Constraints

The input consists of an N -record binary file, where each record has the following structure:

- An 8-byte key used for sorting;
- A 32-bit payload length field;
- A payload of variable size in the range $[8, \text{PAYLOAD_MAX}]$.

The key challenge is that the input file **cannot be assumed to fit in memory**, requiring an external-memory sorting strategy. A strict memory budget is enforced per node and per MPI rank, ensuring that the algorithm remains out-of-core even for large inputs.

3 Common Infrastructure

Our solution relies on a modular set of core utilities shared across all implementations (Sequential, OpenMP, and MPI).

- **Data Representation (record-io.hpp):** To optimize I/O and minimize padding, We use a compact binary record format (**key, length, payload**) and operate on raw byte blocks with helper routines to avoid padding/copies.
- **Streaming I/O (merger.hpp):** To support out-of-core processing, we implemented the **RunStream** class. It manages input files using dynamic buffers (clamped between 256 KB and 64 MB) to stream data continuously from disk without loading the entire file into RAM.
- **Synthetic Data Generation (datagen.cpp):** We developed a datagen tool to create deterministic, reproducible datasets. It generates uniformly distributed 64-bit keys and random payload sizes up to a user-defined PAYLOAD-MAX. In our MPI experiments we used a single shared input file; rank 0 computes record-aligned byte splits at runtime and broadcasts them to all ranks.
- **Merging Kernel (merger.hpp):** The core sorting logic includes two strategies:
 1. **K-Way Merge:** Uses a priority queue (Min-Heap) to merge multiple sorted runs locally in a single pass.
 2. **Implements a buffered multi-way merge (min-heap).** In the MPI reduction phase we reuse the same merge routine with two inputs.
- **Verification (verifier.hpp):** A validation tool performs a linear scan of the final output to ensure non-decreasing ordering (allows equal keys) of keys and that the total record count matches the input size.

3.1 Single-Node Implementation

Before scaling to distributed nodes, we established a robust single-node sorting engine. This implementation serves two purposes: it acts as a performance baseline and functions as the local sorting kernel within each MPI process in the distributed version.

3.1.1 Sequential Baseline

The sequential version implements a standard external merge sort algorithm designed to handle data larger than the physical RAM (`memBytes`). The process is divided into two distinct phases:

1. **Phase 1: Run Generation (In-Memory Sorting)**

- **Reading:** The input file is read in large chunks (blocks) that fit into the defined memory budget (e.g., 64MB or 256MB).
- **Indexing:** Instead of moving heavy payload data, we create a lightweight vector of `offsets` pointing to the start of each record in the raw data block.
- **Sorting:** We use the C++ standard library `std::sort` to sort the `offsets` vector. The comparator function dereferences the offsets to compare the 64-bit keys directly from the raw data buffer.
- **Writing:** The records are written to temporary disk files (e.g., `run_0.bin`, `run_1.bin`) in sorted order. This process repeats until the entire input file has been processed.

2. Phase 2: K-Way Merge

Once all runs are generated, the `mergeFiles` kernel (described in Section 2.1) is invoked. It opens all temporary files simultaneously and merges them into the final `output.bin` using a priority queue.

3.1.2 OpenMP Parallelization

To exploit the multi-core architecture of modern CPUs, we accelerated Phase 1 (Sorting) using OpenMP. The reading, writing, and merging phases remain sequential as they are primarily I/O-bound.

The parallelization strategy replaces `std::sort` with a custom Task-Parallel Merge Sort:

- **Recursive Task Generation:** The sorting range $[L, R)$ is recursively divided into two halves. Each recursive call creates two new OpenMP tasks (`#pragma omp task`) to sort the left and right sub-arrays concurrently.
- **Task Grouping:** We use `#pragma omp taskgroup` to ensure that both child tasks complete before the parent task proceeds to the merge step.
- **Cutoff Strategy:** To prevent excessive task creation overhead, recursion stops when the sub-array size falls below a specific threshold (`cutoff`, typically 2^{15}). At this point, the thread switches to the highly optimized sequential `std::sort`.
- **Parallel Region:** The top-level recursion is launched inside a `#pragma omp parallel` region using a single initial thread (`#pragma omp single`), which then spawns the entire task tree.

This approach allows the sorting workload to be dynamically load-balanced across all available cores, significantly reducing the “Sort Time” component of the execution.

3.1.3 FastFlow Implementation

As an alternative shared-memory approach, we implemented a version using FastFlow, a C++ structured parallel programming framework. Unlike the OpenMP version, which focuses on fine-grained data parallelism (parallelizing the sort of a single buffer), the FastFlow implementation exploits coarse-grained Stream Parallelism using a Farm Pattern.

The sorting pipeline is constructed as follows:

- **Emitter (Source):**

- The Emitter is responsible for sequential I/O. It reads the input file in chunks (`blockBytes`) determined by the memory budget.
- For each chunk, it creates a **Task** containing the raw data vector and dispatches it to the worker pool. This ensures that disk reading overlaps with the computation performed by workers on previous chunks.

- **Farm of Workers (Compute Kernel):**

- A pool of parallel workers receives tasks from the Emitter.
- Each worker operates independently on its assigned data block. It performs the Indexing and Sorting steps (using `std::sort` on offsets) and immediately Writes the sorted run to a unique temporary disk file.
- This design allows multiple distinct operations to occur simultaneously: while one thread sorts Block i , another might be writing Block $i - 1$, and the Emitter is reading Block $i + 1$.

- **Collector (Sink):**

- A simplified Collector gathers completion signals from workers, tracking the total number of runs generated and managing memory deallocation.

3.2 Distributed Implementation (Hybrid MPI + OpenMP)

To scale beyond a single node, we developed a distributed algorithm using **Hybrid Parallelism**. This approach combines **MPI** for inter-node communication and data orchestration with **OpenMP** for intra-node computational acceleration. The execution flow is divided into three distinct phases.

3.2.1 Phase 0: Virtual Data Splitting

Unlike standard MPI **Scatter** operations which move data, our solution leverages the shared file system to minimize initial bandwidth usage.

- **Rank 0 orchestration:** The master process (Rank 0) calculates “virtual splits” by dividing the file size by the number of MPI ranks (P).
- **Boundary Alignment:** A critical challenge in variable-length record formats is ensuring that splits do not slice through the middle of a record. Rank 0 validates the split points by inspecting the file at the calculated offsets. It scans forward byte-by-byte inside a fixed window until it finds an offset where **K consecutive records** have a valid header and length (**bounds + record fits**), ensuring splits do not cut through a record.
- **Broadcast:** The resulting list of start/end offsets is broadcast (`MPI_Bcast`) to all ranks, allowing each node to open the file independently at the correct position.

3.2.2 Phase 1: Local Parallel Sorting

In this phase, all MPI ranks work in parallel without communication.

- **Reading:** Each rank reads its assigned byte range from the shared input file.
- **Sorting:** Inside each rank, we reuse the **OpenMP Task-Parallel** engine (described in Section 3.1.2) to generate sorted runs. This ensures that all available cores on the node are utilized for sorting.
- **Local Merge:** The rank merges its local runs into a single, fully sorted intermediate file (e.g., `sorted_r3.bin`) stored in local scratch space. An `MPI_Barrier` ensures all nodes complete this heavy computational phase before proceeding.

3.2.3 Phase 2: Global Binary Reduction (Merge Tree)

To produce the final output, the distributed local files must be merged. We implement a **Binary Reduction Tree** strategy to perform this efficiently across the cluster.

- **The Algorithm:** In each step k (where $k = 1, 2, 4 \dots$), active ranks are paired.
 - **Sender:** A rank with ID i where $i \pmod{2k} \neq 0$ sends its entire sorted file to rank $i - k$ and retires.
 - **Receiver:** A rank with ID i where $i \pmod{2k} = 0$ receives the file stream from rank $i + k$, merges its current sorted file with the received file by calling `mergeFiles(...)` on two input files, producing a new merged file for the next reduction step, and continues to the next step.
- **Streaming Transfer:** To handle files larger than RAM, we implemented custom `send_file_stream` and `recv_file_stream` functions. These break the file into 32 MB chunks and transmit them using standard `MPI_Send` and `MPI_Recv` loops, preventing MPI buffer overflows.
- **Result:** After $\log_2(P)$ steps, Rank 0 holds the globally sorted dataset.

4 Performance Evaluation

4.1 Experimental Setup

All experiments were conducted on the SPM cluster:

- Multi-core compute nodes connected via 10 GbE;
- SLURM used for resource management with exclusive node allocation;
- Memory limits enforced explicitly to guarantee out-of-core execution.

Datasets were generated with varying numbers of records and payload sizes to explore both CPU-bound and I/O-bound scenarios.

4.2 Single Node Performance Evaluation

4.2.1 Thread Scalability (Intra-Node Parallelism)

The first set of experiments evaluates scalability with respect to the number of parallel workers on a single node. The problem size and memory budget were kept fixed, while the number of OpenMP threads or FastFlow workers was increased.

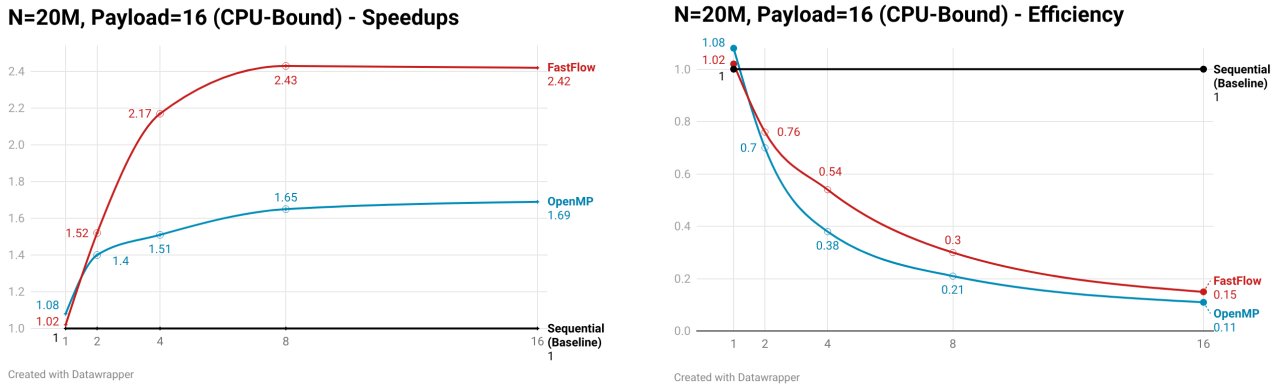
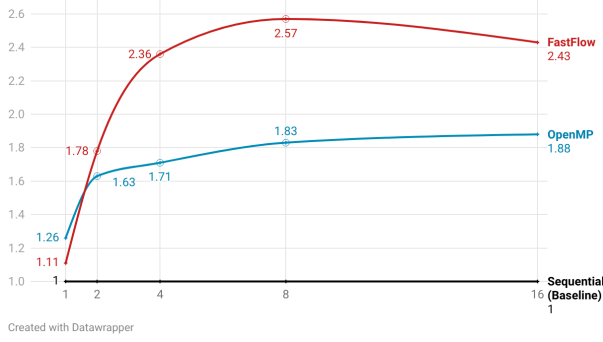


Figure 1: Memory was set to 64 MB. OpenMP achieves $\sim 1.7x$ speedup by parallelizing the CPU-bound sort but plateaus quickly due to strict serial I/O and merge phases (Amdahl’s Law). FastFlow reaches $\sim 2.4x$ by using a farm pattern to pipeline reads and parallelize writes, effectively hiding latency until the disk bandwidth hits saturation. Efficiency drops steeply for both as the unoptimized sequential Merge phase becomes the dominant bottleneck, rendering additional threads/workers ineffective.

N=20M, Payload=128 (I/O-Bound) - Speedups



N=20M, Payload=128 (I/O-Bound) - Efficiency

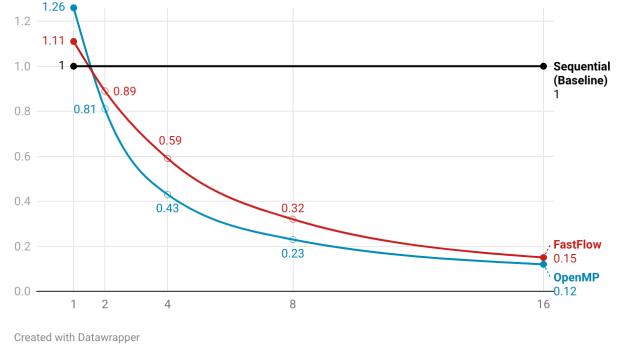
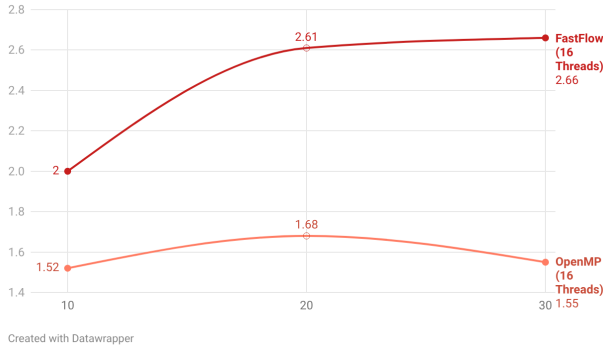


Figure 2: In this I/O-heavy scenario (128B payload, RAM=256MB), OpenMP’s gains are severely limited ($\sim 1.9x$) as it leaves the dominant Read/Write/Merge phases serialized. FastFlow achieves higher speedup ($\sim 2.6x$) by parallelizing the massive write load, but performance degrades beyond 8 workers due to disk thrashing (seeking contention) caused by too many concurrent file streams. Both implementations hit a hard floor at $\sim 3.2s$ due to the strictly sequential merge phase.

4.2.2 Problem Size Scaling (Varying N)

To study robustness with respect to input size, experiments were conducted by varying the number of records (e.g., 10M, 20M, 30M) while keeping the memory budget and number of workers fixed.

N =10M,20M,30M Payload=16 (CPU-Bound) - Speedups



N=10M,20M,30M - Payload= 16 (CPU-Bound) - Efficiency

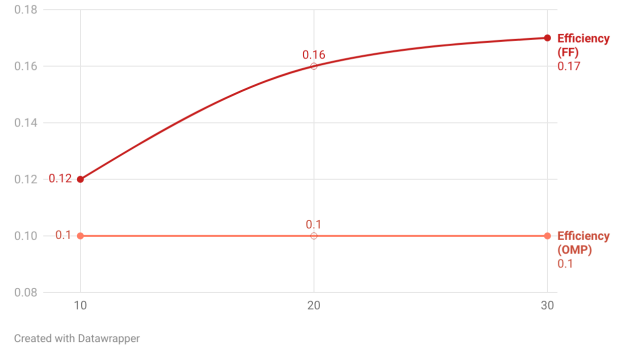
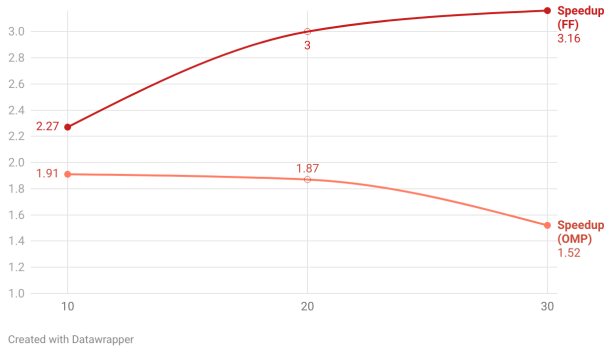


Figure 3: In the CPU-bound scenario (Payload=16), Sequential execution scales linearly with input size. OpenMP yields a constant $\sim 1.6x$ speedup but is severely limited by the sequential Write/Merge phases, which constitute $\sim 67\%$ of the 30M runtime. FastFlow achieves superior scalability ($\sim 2.6x$) by parallelizing writes, though overhead degrades performance at 16 workers for the smallest (10M) dataset. Both implementations remain ultimately bounded by the strictly sequential Merge phase.

N = 10M, 20M, 30M Payload=128 (I/O-Bound) - Speedups



N = 10M, 20M, 30M - Payload= 128 (I/O-Bound) - Efficiency

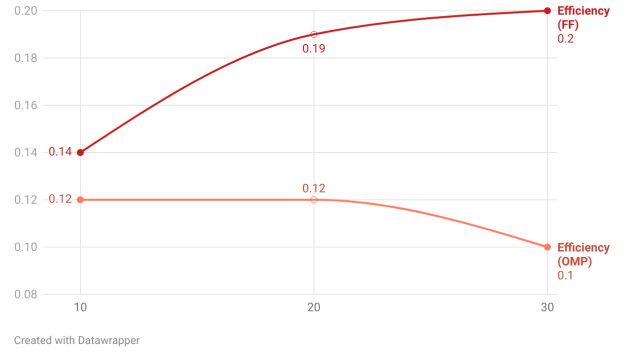


Figure 4: Scalability vs. Input Size. As the input size scales from 10M to 30M records, OpenMP’s speedup degrades (dropping to $\sim 1.5x$) because the strictly sequential Write and Merge phases grow linearly, dominating the execution time. In contrast, FastFlow maintains robust scalability ($\sim 3.1x$ speedup) by parallelizing the massive I/O load, though it eventually becomes bounded by the sequential Merge phase, which consumes $> 50\%$ of total runtime for the largest dataset.

4.2.3 Payload Sensitivity (CPU-Bound vs I/O-Bound)

The impact of payload size was evaluated by comparing workloads with small payloads (CPU-oriented) and large payloads (I/O-oriented), keeping N, memory budget, and parallelism fixed.

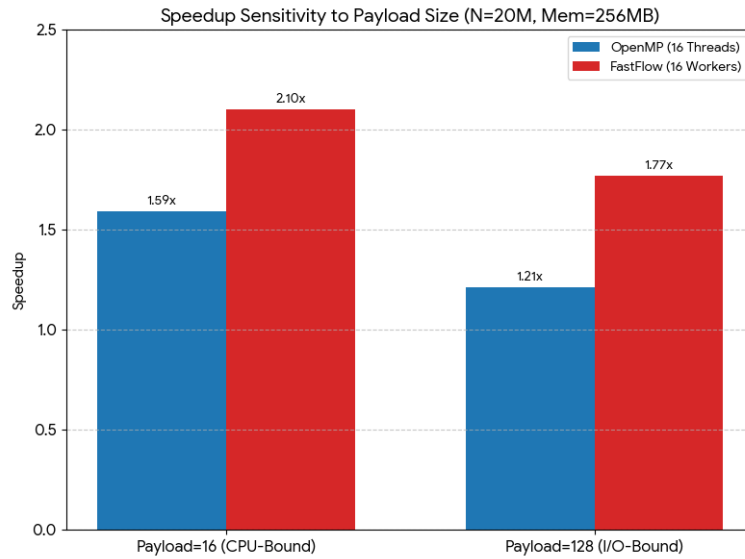


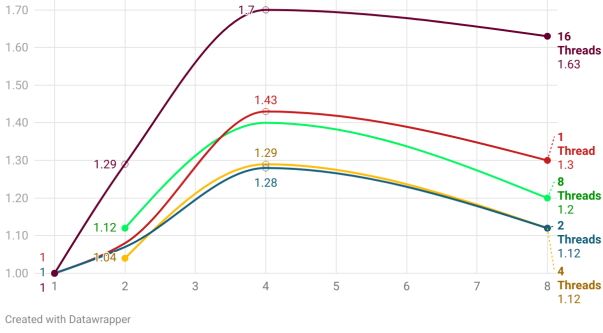
Figure 5: Payload Sensitivity Analysis (N=20M, Mem=256MB). Increasing the payload size from 16 to 128 bytes triples the data volume, shifting the bottleneck heavily toward Disk I/O. OpenMP’s performance degrades significantly ($1.59x \rightarrow 1.21x$) because its write phase is strictly sequential. In contrast, FastFlow demonstrates superior robustness ($2.10x \rightarrow 1.77x$) by parallelizing the massive write load across 16 workers, although both implementations ultimately face the limit of the unoptimized sequential merge phase.

4.3 Multinode Performance Evaluation

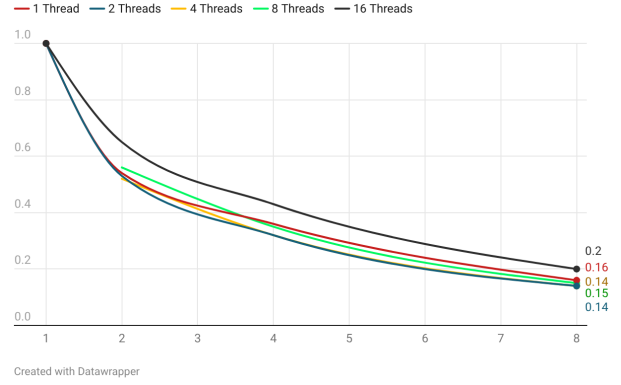
4.3.1 MPI Strong Scaling (Internode)

We evaluated strong scaling by fixing the total dataset size at 30 million records and increasing the number of nodes from 1 to 8, aiming to measure the reduction in execution time as computational resources are added to a fixed workload.

Varying Threads - Across 8 Nodes (CPU-Bound) - Speedups



Varying Threads - Across 8 Nodes (CPU-Bound) - Efficiency



Varying Threads - Across 8 Nodes - Absolute Time

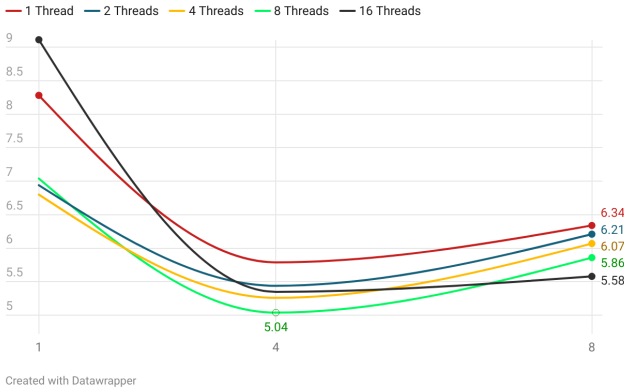


Figure 6: Strong Scaling Analysis (N=30M). This reveals that Hybrid MPI+OpenMP (16 Threads) achieves the best relative scalability ($\sim 1.7x$) by minimizing MPI ranks and communication overhead. However, performance peaks at 4 nodes for all configurations and degrades at 8 nodes, indicating the problem size is too small to justify further expansion (communication dominates computation). While the 16-thread hybrid approach scales most efficiently, the 8-thread configuration delivers the fastest absolute execution time (5.04s) at the 4-node peak.

4.3.2 MPI Weak Scaling (Internode)

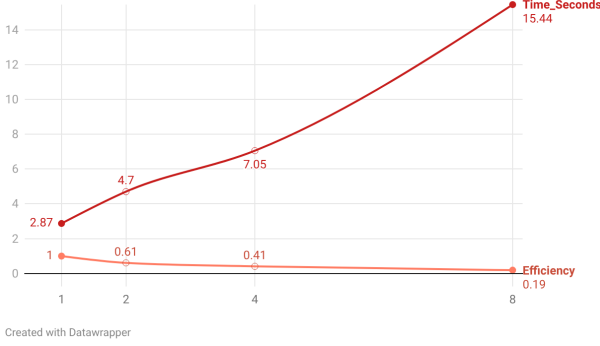
We assessed weak scaling by maintaining a constant workload of 10 million records per node (increasing the total dataset from 10M to 80M) to evaluate the system's efficiency in handling larger problem sizes while maintaining stable execution.

The analysis of the weak scaling is as follows:

- **Local Computation (Phase 1):** Scaled near-perfectly. The sorting time increased only slightly from 2.86s (1 node) to 3.72s (8 nodes), demonstrating that the $O(N \log N)$ local sort is well-parallelized.

- **Global Communication (Phase 2):** Exhibited poor scaling. The merge time surged from near-zero to 11.59s at 8 nodes. Consequently, the **Weak Scaling Efficiency** dropped to 19%, and the **Scaled Speedup** saturated at 1.49x (instead of the ideal 8x). This confirms that the network bandwidth during the reduction-tree global merge phase (Phase 2) is the primary scalability bottleneck.

Weak Scaling Efficiency Across 8 Nodes



Formula: $ScaledSpeedup = Nodes \times \frac{Time_{1Node}}{Time_{Nnodes}}$

Nodes	Total Workload	Execution Time	Scaled Speedup (Ideal = Nodes)	Efficiency (Ideal = 1.0)
1	10M	2.87s	1.00	1.00
2	20M	4.70s	1.22	0.61
4	40M	7.05s	1.63	0.41
8	80M	15.44s	1.49	0.19

Figure 7: Ideally, with 8 nodes, we should achieve a **Scaled Speedup** of 8.0 (meaning we processed 8x the data at the same time). However, we achieved a speedup of only 1.49. This indicates that while we added 8x the compute power, the network overhead reduced our effective throughput gain to just $\sim 1.5x$.

5 Analytical Cost Model

To interpret the experimental results, we utilize a simplified cost model that decomposes the execution into local sorting (Phase 1) and global merging (Phase 2).

5.1 Cost Model (MPI + Shared-Memory)

1. Local Out-of-Core Sort (Phase 1)

Each rank processes approximately $1/P$ of the input. Let S be the total input bytes, P the number of MPI ranks, and T the number of threads/workers per rank. The local execution time is dominated by disk I/O passes and CPU sorting:

$$T_{\text{local}} \approx \underbrace{\frac{S}{P \cdot BW_{\text{disk}}}}_{\text{read input}} + \underbrace{\frac{S}{P \cdot BW_{\text{disk}}}}_{\text{write runs}} + \underbrace{\frac{S}{P \cdot BW_{\text{disk}}}}_{\text{read runs}} + \underbrace{\frac{S}{P \cdot BW_{\text{disk}}}}_{\text{write local sorted}} + \underbrace{\frac{N \log(N/P)}{P \cdot T}}_{\text{CPU sorting}} \quad (1)$$

Interpretation: The data passes through the disk approximately 4 times (run generation + local merge). While increasing threads (T) reduces the CPU sorting term, it does not affect the dominant disk I/O terms. This explains the single-node results where speedup saturates once I/O and merge operations dominate.

2. Global Merge (Phase 2)

The P locally-sorted files are merged using a reduction tree, requiring $\log_2 P$ steps. At each step, data is exchanged over the network and merged on disk:

$$T_{\text{global}} \approx \underbrace{\frac{S \log P}{BW_{\text{net}}}}_{\text{send/recv large files}} + \underbrace{\frac{S \log P}{BW_{\text{disk}}}}_{\text{re-read + re-write during merges}} \quad (2)$$

Interpretation: As the number of nodes increases, the number of merge levels grows. Since each level involves moving and rewriting the entire dataset, the global merge cost scales with $S \log P$, becoming the primary bottleneck at scale. This aligns with the observed "weak scaling collapse" at 8 nodes.

3. Total Execution Time

The total time is determined by the slowest local sort plus the global merge overhead:

$$T_{\text{total}} \approx \max(T_{\text{local}}) + T_{\text{global}} \quad (3)$$

6 Discussion of Results

6.1 Observed Bottlenecks

- **Single-Node:**
 - **CPU-Bound Payloads:** Sorting scales well initially, but memory bandwidth and the sequential merge phase eventually cap speedup.
 - **I/O-Bound Payloads:** Disk access dominates; adding more threads yields diminishing returns. FastFlow outperforms OpenMP here by overlapping I/O and computation (pipeline parallelism), though it is still ultimately bounded by disk bandwidth.
- **Multi-Node:**
 - **Phase 1** scales efficiently as it is embarrassingly parallel.
 - **Phase 2** (Global Merge) creates the bottleneck due to heavy network transfers, multiple extra disk passes, and the serial dependency of the reduction tree.

6.2 Computation-to-Communication Overlap Effectiveness

- **Effective Overlap (Single-Node FastFlow):** In the single-node implementation, we achieve I/O-compute overlap during the run generation phase. The pipeline/farm architecture allows the system to read the next data block while concurrently sorting the

current block and writing the previous one. This architectural advantage is the primary reason FastFlow beats OpenMP in I/O-heavy workloads.

- **Ineffective Overlap (Distributed MPI Phase):** In the distributed Global Merge (Phase 2), there is **no** overlap between communication and merging.
 - The receiver logic is sequential: it downloads the entire intermediate file chunk, writes it to disk, and only then proceeds to merge.
 - A strict synchronization barrier exists between Phase 1 and Phase 2.

Consequently, the overlap effectiveness in Phase 2 is near zero. This explains the collapse in weak scaling: Phase 2 is effectively a serialized sequence of "move data → rewrite data → merge," without hiding the communication cost.

6.3 Implementation Challenges

- **Variable-Length Records:** Ensuring split points did not slice through records required byte-by-byte scanning for valid headers.
- **Strict Memory Budget:** Guaranteeing out-of-core execution required precise buffer management to prevent OOM errors.
- **Merge Cost Explosion:** As N increased, the number of temporary runs grew, increasing the cost of the final merge pass.
- **Disk Contention:** High worker counts in FastFlow occasionally caused "thrashing" due to too many concurrent file streams.

6.4 Adopted Optimizations

- **Offset Sorting:** Sorted pointers/offsets rather than moving heavy payload data in memory.
- **Task Cutoff:** Stopped OpenMP recursion early to switch to `std::sort`, avoiding task creation overhead.
- **Buffered I/O:** Implemented custom buffering to minimize syscall overhead.
- **Tree Merge:** Used a binary reduction tree for global merging instead of a single bottlenecked Gather operation.
- **Chunked Transfers:** Broke large files into small chunks for MPI transmission to prevent buffer overflows and ensure stability.

6.5 Limitation

Parallel merge. The external merge was implemented as a deterministic 2-way merge in the MPI reduction tree (and a k-way merge locally), without additional intra-merge multithreading. This choice is intentional: in the measured workloads the merge is dominated by disk bandwidth and repeated file passes, so adding threads inside merge would mainly increase I/O contention. Moreover, a fully parallel external merge would require extra buffering and more complex partitioning/indexing to split merge work among threads while respecting the strict memory budget and variable-length record format. For this project we prioritized correctness, bounded memory usage, and reproducible behavior on the cluster.

7 Future Work

To address the bottlenecks identified in the distributed phase, future optimizations could include:

- **Stream-Merge and Advanced Partitioning:** Implementing a stream-merge strategy during reception would enable true overlap between communication and merging. Additionally, parallelizing the merge phase using range partitioning (with sampled splitters) could further reduce Phase 2 latency, provided sufficient memory is available for the additional buffering required.

8 Conclusion

This project delivers a complete out-of-core MergeSort pipeline that remains correct and stable under strict memory budgets, and scales from a single node to multiple nodes by combining shared-memory parallelism with distributed execution.

On a single node, FastFlow consistently outperforms the OpenMP version because its pipeline/farm structure overlaps I/O with computation and reduces scheduling overhead, although both approaches eventually saturate when disk bandwidth and the external merge phase dominate.

In the distributed setting, the MPI reduction-tree merge provides a simple and robust way to compose locally sorted partitions into a globally sorted output, showing good scaling up to moderate node counts; beyond that, performance is increasingly limited by global merge costs (network transfers and additional disk passes) rather than CPU sorting.

Overall, the implementation demonstrates that structured parallel programming and careful memory-bounded external sorting can achieve practical performance while staying defensible and reproducible; future improvements would focus on reducing the global-merge bottleneck, e.g., by streaming merge during reception and exploring more overlap-aware merging strategies.